

Programlamaya Giriş

Bilgisayar ve Programlamaya Giriş

Diller

Çeviri

Algoritma

Yazılım süreci

Temel yapılar

Sercan KÜLCÜ · Giresun Üniversitesi

Öğrenme Çıktıları

Kavramlar

Kaynak kod, makine dili ve programlama dilini ayırt etmek.

Çalıştırma

Derleme, yorumlama ve hibrit çalışma biçimlerini açıklamak.

Problem çözme

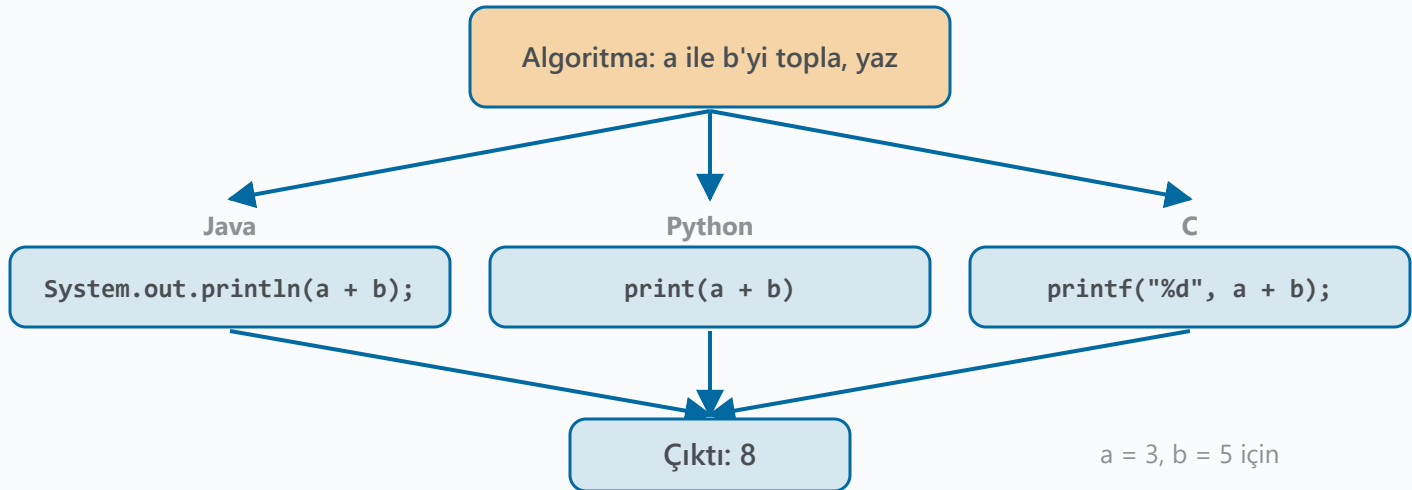
Algoritmadan programa giden süreci kurmak.

Yapılar

Değişken, karar, döngü, fonksiyon ve diziyi tanımak.

Programlama Dili Nedir?

- İnsanların bilgisayara işlem adımlarını ifade etmesini sağlayan biçimsel bir dildir.
- Söz dizimi** (syntax) hangi yapıların yazılabileceğini, **anlam** (semantics) bu yapıların ne yaptığını belirler.
- Aynı problem farklı dillerle çözülebilir; seçim gereksinim ve ortama bağlıdır.



Kaynak Kod ve Makine Dili

Kaynak kod

insan okur, yazar

```
toplam = a + b;
```

derleyici çevirir

Assembly

komutların simgesel adı

```
LOAD R1, a · ADD R1, b · STORE R1, toplam
```

assembler çevirir

Makine dili

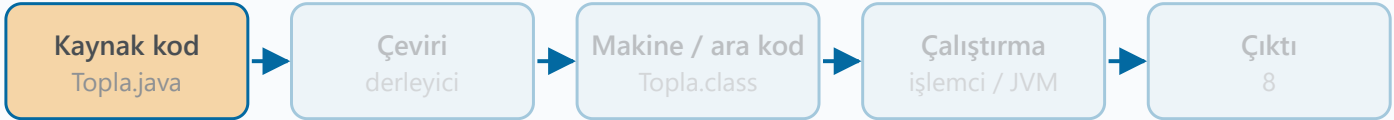
işlemci çalıştırır (ikili)

```
0001 0001 1000 · 0011 0001 1001 · 0010 0001 1010
```

Makine kodu örnektir (gerçek kodlama işlemciye göre değişir).

- Kaynak kod geliştiricinin okuyup değiştirebildiği program metnidir; makine dili işlemcinin çalıştırdığı ikili komutlardır.
- Derleyici veya yorumlayıcı aradaki köprüyü kurar: programlama dili ile işlemci dili aynı şey değildir.

Program Nasıl Çalışır?

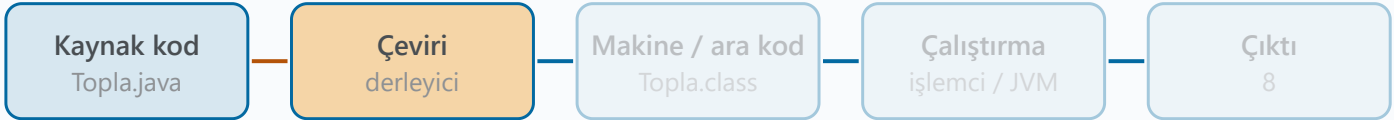


Programcı **kaynak kodu** bir metin dosyasına yazar. Bilgisayar bu metni doğrudan çalıştıramaz.

Adım 1 / 5

Hata türleri: **derleme** (söz dizimi kuralı), **çalışma zamanı** (yürütme sırasında), **mantık** (yanlış sonuç).

Program Nasıl Çalışır?



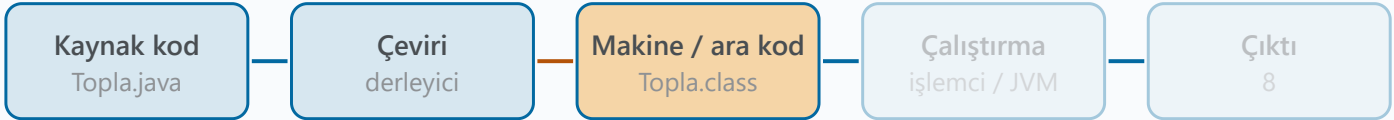
✗ **derleme hatası**

Çeviri: derleyici söz dizimini denetler. Noktalı virgül unutulursa **derleme hatası** burada çıkar, program hiç çalışmaz.

Adım 2 / 5

Hata türleri: **derleme** (söz dizimi kuralı), **çalışma zamanı** (yürütme sırasında), **mantık** (yanlış sonuç).

Program Nasıl Çalışır?

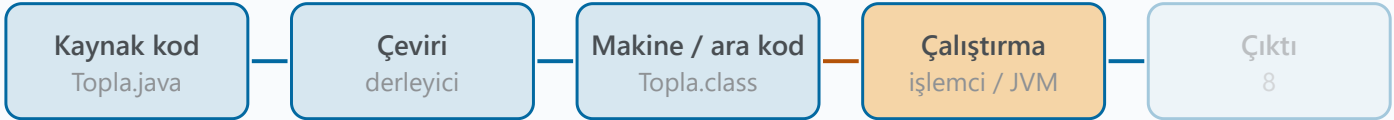


Hata yoksa **makine kodu** ya da Java'da olduğu gibi **ara kod** (bytecode) üretilir.

Adım 3 / 5

Hata türleri: **derleme** (söz dizimi kuralı), **çalışma zamanı** (yürütme sırasında), **mantık** (yanlış sonuç).

Program Nasıl Çalışır?



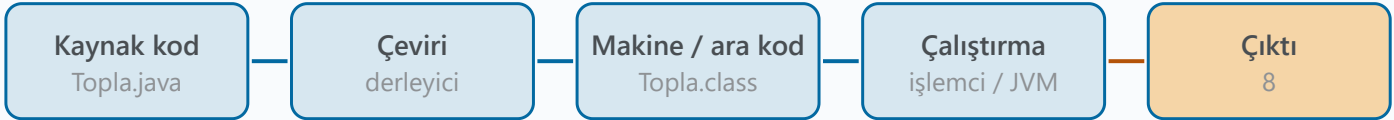
X çalışma zamanı hatası

Çalıştırma: komutlar işlemcide (Java'da JVM üzerinden) yürütülür. 0'a bölme gibi **çalışma zamanı hataları** burada ortaya çıkar.

Adım 4 / 5

Hata türleri: **derleme** (söz dizimi kuralı), **çalışma zamanı** (yürütme sırasında), **mantık** (yanlış sonuç).

Program Nasıl Çalışır?



Çıktı, verilen girdiye ($a = 3$, $b = 5$) ve program mantığına bağlıdır. Mantık hatası varsa program çalışır ama yanlış sonuç verir.

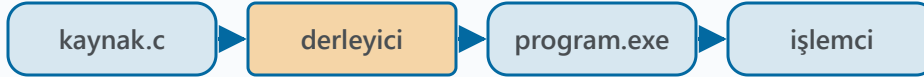
Adım 5 / 5

Hata türleri: **derleme** (söz dizimi kuralı), **çalışma zamanı** (yürütme sırasında), **mantık** (yanlış sonuç).

Çalıştırma Yaklaşımları

Yaklaşım	İşleyiş	Örnek
Derlenen	Program önceden hedef makine koduna çevrilir	C, C++, Rust
Yorumlanan	Komutlar çalışma sırasında değerlendirilir	Python, JavaScript
Hibrit	Ara kod sanal makinede yürütülür	Java, C#

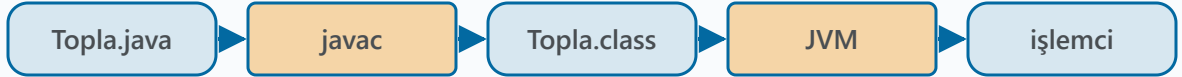
Derlenen



Yorumlanan



Hibrit (Java)



turuncu: çeviri aracı

Derleyici ve Yorumlayıcı Adım Adım

Derleyici: önce tümü çevrilir, sonra çalışır

- | | | |
|---|---------------|------------|
| 1 | yaz "Merhaba" | çevriliyor |
| 2 | x = 10 | çevriliyor |
| 3 | yaz x / 0 | çevriliyor |

konsol



Derleyici programın **tamamını** baştan sona okur; henüz hiçbir satır çalışmadı.

Adım 1 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Derleyici: önce tümü çevrilir, sonra çalışır

1 yaz "Merhaba" ✓

2 x = 10 ✓

3 yaz x / 0 ✓

konsol



Satır 3'teki 0'a bölme söz dizimi hatası değildir; çeviri başarılı olur ve çalıştırılabilir dosya üretilir.

Adım 2 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Derleyici: önce tümü çevrilir, sonra çalışır

1 `yaz "Merhaba"` ► çalışıyor

2 `x = 10` ✓

3 `yaz x / 0` ✓

konsol

Merhaba

Şimdi çalıştırma başlar: satır 1 → çıktı **Merhaba**.

Adım 3 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Derleyici: önce tümü çevrilir, sonra çalışır

- 1 yaz "Merhaba" ✓
- 2 x = 10 ✓
- 3 yaz x / 0 ✗ hata

konsol

```
Merhaba  
HATA
```

Satır 3 çalışırken **çalışma zamanı hatası** (0'a bölme). Derleyicinin açıklayamadığı hata türü.

Adım 4 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Yorumlayıcı: satır satır çevir ve çalıştır

1 `yaz "Merhaba"` ► çalışıyor

2 `x = 10`

3 `yaz x / 0`

konsol

Merhaba

Yorumlayıcı ise **çevir-çalıştır** döngüsüyle ilerler: satır 1 çevrilir ve hemen çalışır → Merhaba.

Adım 5 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Yorumlayıcı: satır satır çevir ve çalıştır

- 1 `yaz "Merhaba"` ✓
- 2 `x = 10` ► çalışıyor
- 3 `yaz x / 0`

konsol

Merhaba

Satır 2 çevrilip çalışır: **x = 10**. Satır 3 henüz hiç okunmadı.

Adım 6 / 7

Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Derleyici ve Yorumlayıcı Adım Adım

Yorumlayıcı: satır satır çevir ve çalıştır

- 1 yaz "Merhaba" ✓
- 2 x = 10 ✓
- 3 yaz x / 0 ✗ hata

konsol

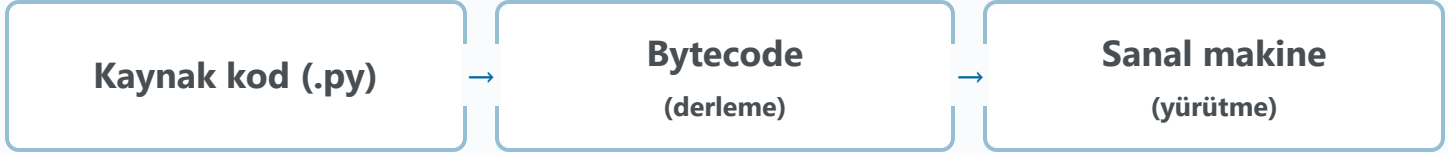
```
Merhaba  
HATA
```

Satır 3'e gelince hata. Önceki satırlar **çoktan çalışmıştı**. Basit, satır satır çalışan bir yorumlayıcıda bu hata da ancak o satıra gelince fark edilirdi.

Adım 7 / 7

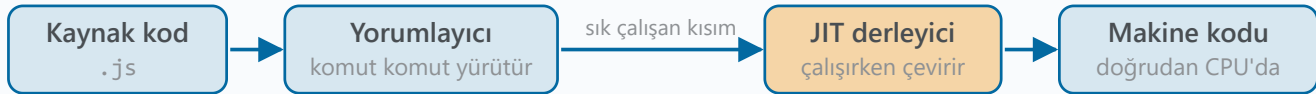
Derleyici söz dizimi hatalarını program çalışmadan **önce** yakalar; yorumlayıcı hemen başlar; hatayı (basit yorumlayıcılarda) o satıra gelince görür. Python ve JavaScript gibi modern yorumlayıcılar önce tüm dosyayı ayrıştırır ve söz dizimi hatasını en başta bildirir; Python ayrıca bytecode'a derler.

Python Neden “Yorumlanan” Denir?



- **CPython** önce kaynak kodu **bytecode**'a derler, sonra bunu bir sanal makinede komut komut yürütür; makine kodu dosyası üretilmez.
- “Yorumlanan” etiketi bu yürütme biçiminden gelir; derleme adımı programcıdan gizlidir.
- Modern **JavaScript** motorları (ör. V8) **JIT** kullanır: sık çalışan kodu çalışma sırasında makine koduna çevirir.

JavaScript motoru (ör. V8): önce yorumlar, sık çalışan kısmı JIT ile makine koduna çevirir

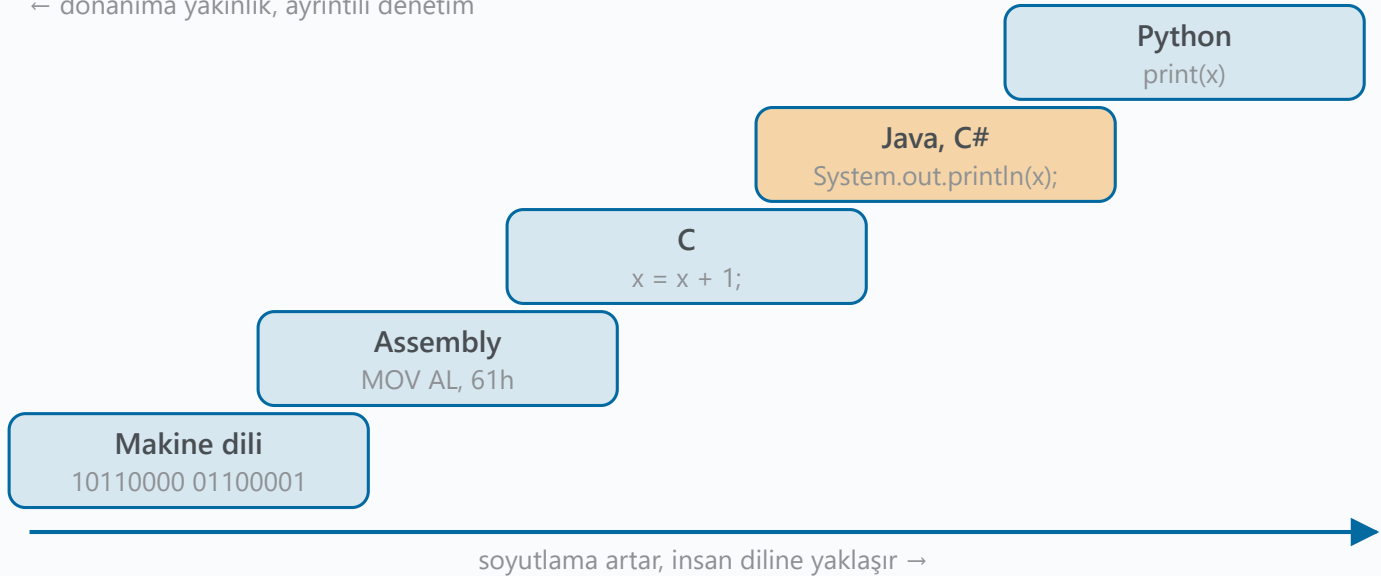


Sonraki çağrılarda yorumlama atlanır; program çalıştıkça hızlanır.

Derlenen / yorumlanan ayrımı dilin değil, **uygulamanın** özelliğidir; pratikte çoğu sistem iki yöntemi birleştirir.

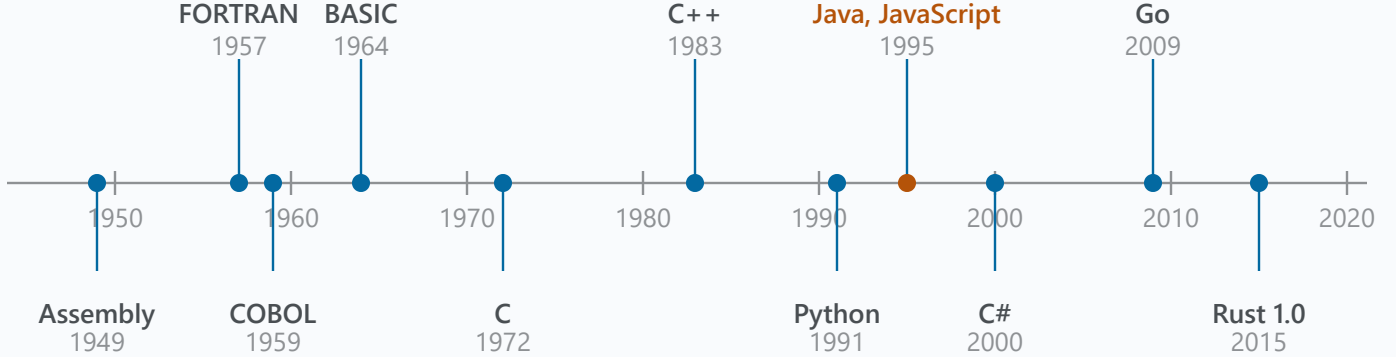
Programlama Dillerinin Seviyesi

← donanıma yakınlık, ayrıntılı denetim



- **Düşük seviyeli** diller donanıma yakındır; **yüksek seviyeli** diller problemi insanın ifade biçimine yakın yazdırır.
- Seviye, bir dilin iyi veya kötü olduğunu değil, **soyutlama düzeyini** anlatır.

Programlama Dillerinin Gelişimi



1940–1960

Makine dili, assembly; ilk yüksek seviyeli diller FORTRAN ve COBOL.

1970–1990

C ile yapısal ve taşınabilir sistem programlama; C++ ile nesne yönelimli yaklaşım.

1990 sonrası

Python, Java, JavaScript, C#: web, yönetilen ortamlar, açık kaynak ekosistemleri.

Kullanım Alanları

Web

İstemci, sunucu ve web servisleri.

Mobil

Telefon ve tablet uygulamaları.

Veri ve yapay zekâ

Analiz, modelleme ve otomasyon.

Sistem

İşletim sistemi, sürücü ve gömülü yazılım.

Bilimsel hesaplama

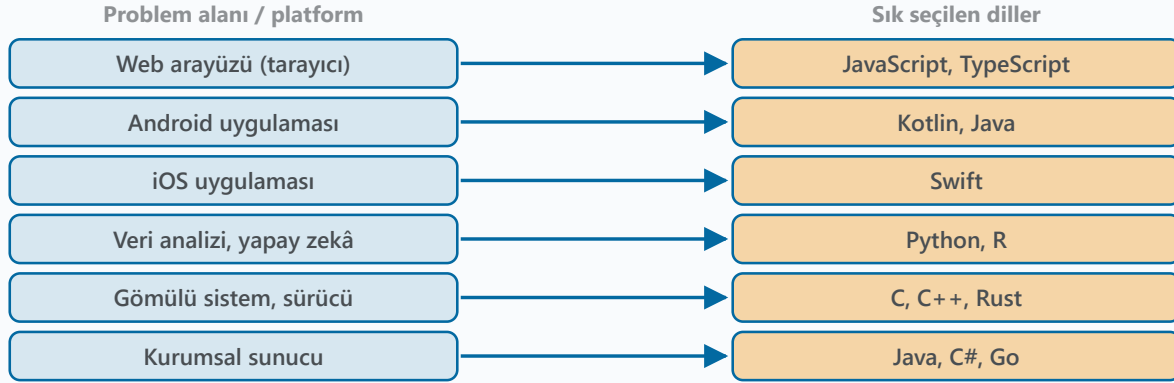
Simülasyon ve mühendislik çözümleri.

Oyun

Oyun motoru, grafik ve etkileşim.

Dil Seçerken

- Problem alanı ve kullanılacak platform değerlendirilir.
- Kütüphane ekosistemi, topluluk ve bakım maliyeti dikkate alınır.
- Performans, güvenlik, ekip deneyimi ve dağıtım koşulları birlikte düşünülür.

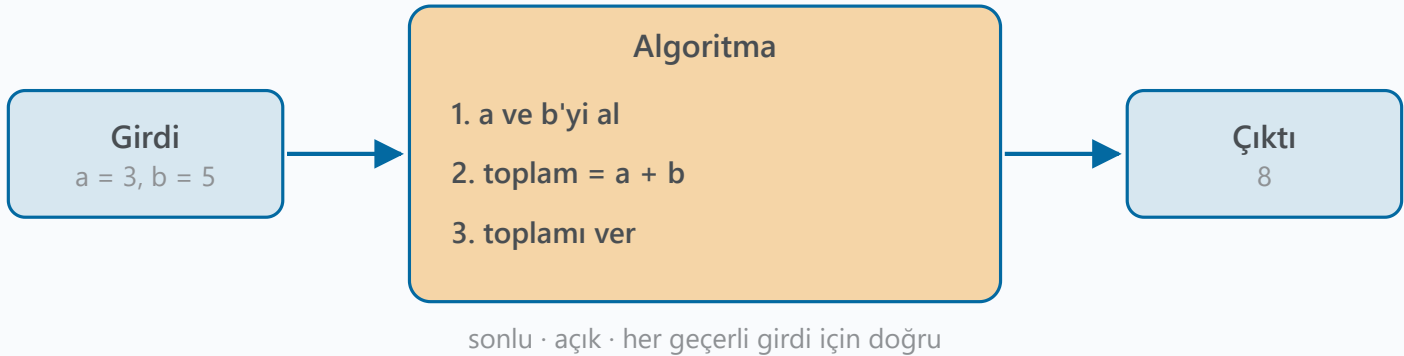


Son kararı kütüphane ekosistemi, performans, güvenlik, ekip deneyimi ve bakım maliyeti verir.

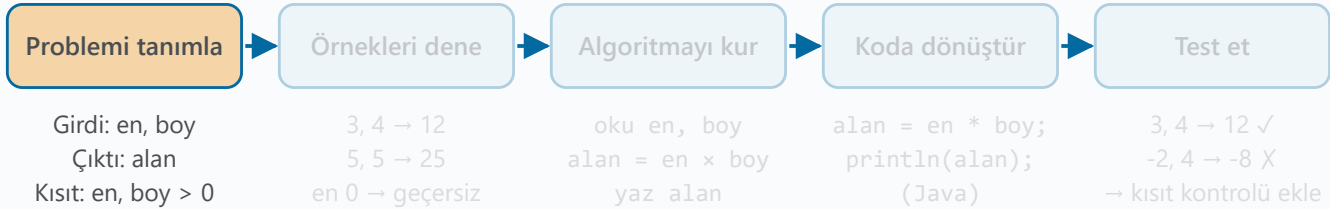
Popülerlik tek başına doğru dil seçimi için yeterli değildir.

Algoritma

- Belirli bir problemi **sonlu** ve **açık** adımlarla çözen yöntemdir.
- Girdiler, beklenen çıktılar ve özel durumlar koddan önce belirlenir.
- Bir algoritma farklı programlama dillerinde uygulanabilir.



Problemden Programa

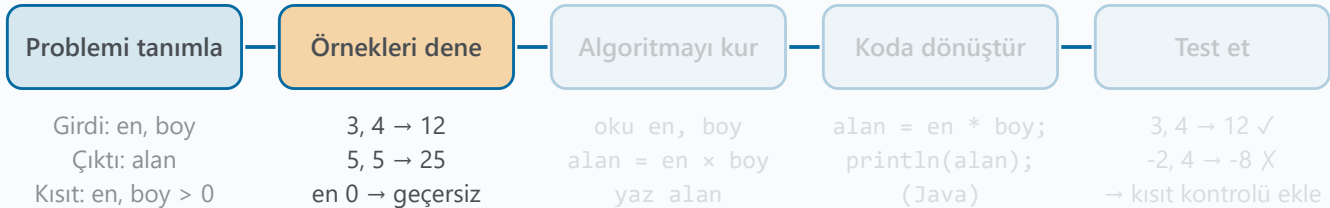


Önce **ne** istendiği netleşir: dikdörtgenin alanı. Girdi, çıktı ve kısıt yazılır.

Adım 1 / 5

- Her aşamanın çıktısı bir sonraki aşamanın girdisidir.
- Yanlış tanımlanan problem, doğru çalışan ama yanlış işi yapan programa dönüşebilir.

Problemden Programa

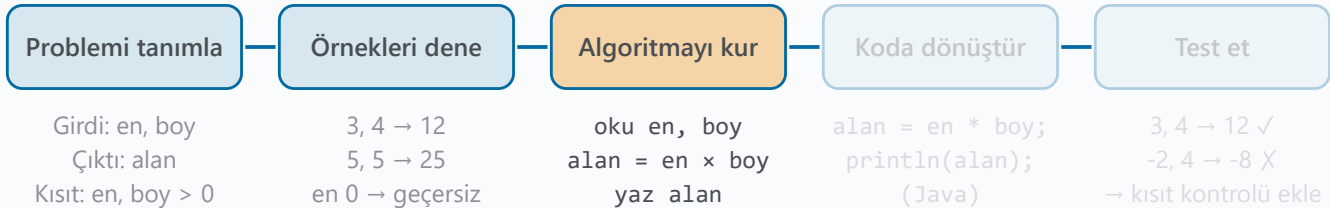


Elle birkaç **örnek** çözülür; sınır durumu (0) da düşünülür.

Adım 2 / 5

- Her aşamanın çıktısı bir sonraki aşamanın girdisidir.
- Yanlış tanımlanan problem, doğru çalışan ama yanlış işi yapan programa dönüşebilir.

Problemden Programa

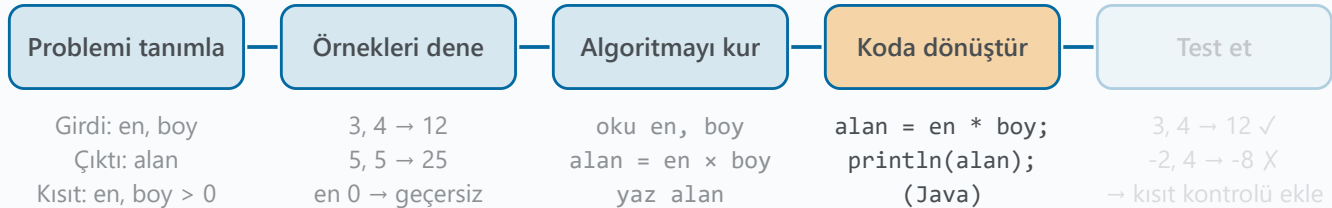


Dilden bağımsız **algoritma** (sözde kod) yazılır.

Adım 3 / 5

- Her aşamanın çıktısı bir sonraki aşamanın girdisidir.
- Yanlış tanımlanan problem, doğru çalışan ama yanlış işi yapan programa dönüşebilir.

Problemden Programa

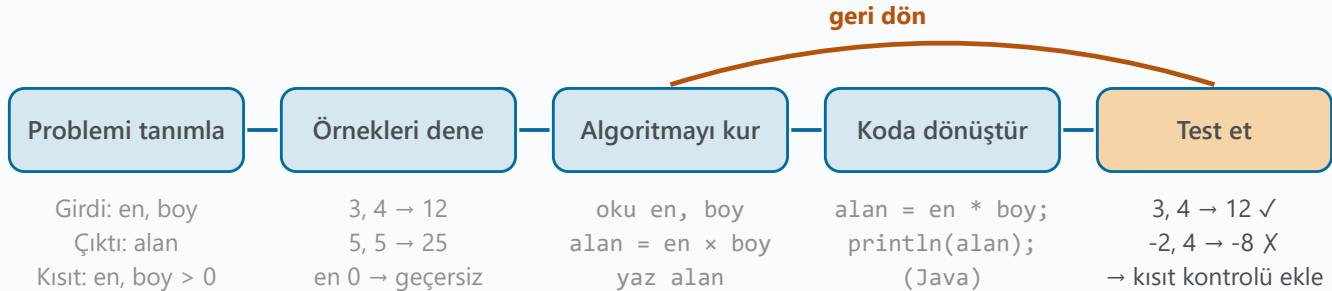


Algoritma seçilen dile (Java) **çevrilir**.

Adım 4 / 5

- Her aşamanın çıktısı bir sonraki aşamanın girdisidir.
- Yanlış tanımlanan problem, doğru çalışan ama yanlış işi yapan programa dönüşebilir.

Problemden Programa



Test: negatif girdi yanlış sonuç verdi → turuncu ok: algoritmaya dönülür, "en, boy > 0 değilse hata" adımı eklenir.

Adım 5 / 5

- Her aşamanın çıktısı bir sonraki aşamanın girdisidir.
- Yanlış tanımlanan problem, doğru çalışan ama yanlış işi yapan programa dönüşebilir.

Algoritmayı İzleme: Dikdörtgen Alanı



1. **çalıştırma**: **en = 3** okunur; bellekte "en" adlı kutuya yazılır.

Adım 1 / 9

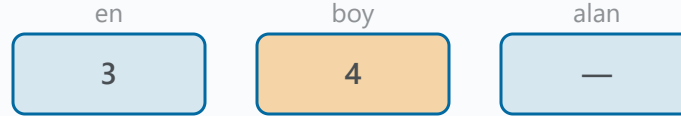
```
► oku en
2 oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = 3

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



boy = 4 okunur.

Adım 2 / 9

```
1 oku en
2 ► oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = 3

boy = 4

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



Koşul: $3 > 0$ ve $4 > 0 \rightarrow$ **doğru**.

Adım 3 / 9

```
1 oku en
2 oku boy
▶ eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = 3

boy = 4

ÇIKTI



Algoritmayı İzleme: Dikdörtgen Alanı



alan = 3 × 4 = 12.

Adım 4 / 9

```
1 oku en
2 oku boy
3 eğer en > 0 ve boy > 0 ise
    ► alan = en × boy
5 yaz "Alan: ", alan
6 değilse
7 yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = 3

boy = 4

alan = 12

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



Sonuç yazılır. "değilse" dalı bu çalıştırmada atlanır.

Adım 5 / 9

```
1 oku en
2 oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     ► yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = 3

boy = 4

alan = 12

ÇIKTI

Alan: 12

Algoritmayı İzleme: Dikdörtgen Alanı



2. **çalıştırma**: **en** = -2 okunur; bellekte "en" adlı kutuya yazılır.

Adım 6 / 9

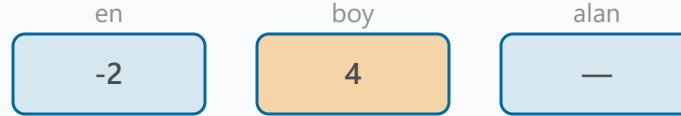
```
► oku en
2 oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = -2

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



boy = 4 okunur.

Adım 7 / 9

```
1 oku en
2 ► oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = -2

boy = 4

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



Koşul: **-2 > 0 ve 4 > 0** → **yanlış** (en negatif) → değilse dalına geçilir.

Adım 8 / 9

```
1 oku en
2 oku boy
3 ► eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
7     yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = -2

boy = 4

ÇIKTI

Algoritmayı İzleme: Dikdörtgen Alanı



Uyarı yazılır; alan hiç hesaplanmadı. Test aşamasında bulunan hata böylece düzeltildi.

Adım 9 / 9

```
1 oku en
2 oku boy
3 eğer en > 0 ve boy > 0 ise
4     alan = en × boy
5     yaz "Alan: ", alan
6 değilse
    ► yaz "Geçersiz ölçü"
```

DEĞİŞKENLER

en = -2

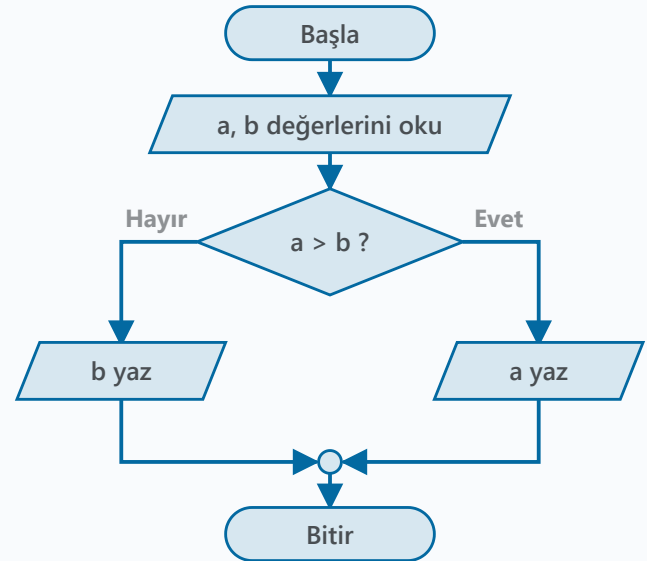
boy = 4

ÇIKTI

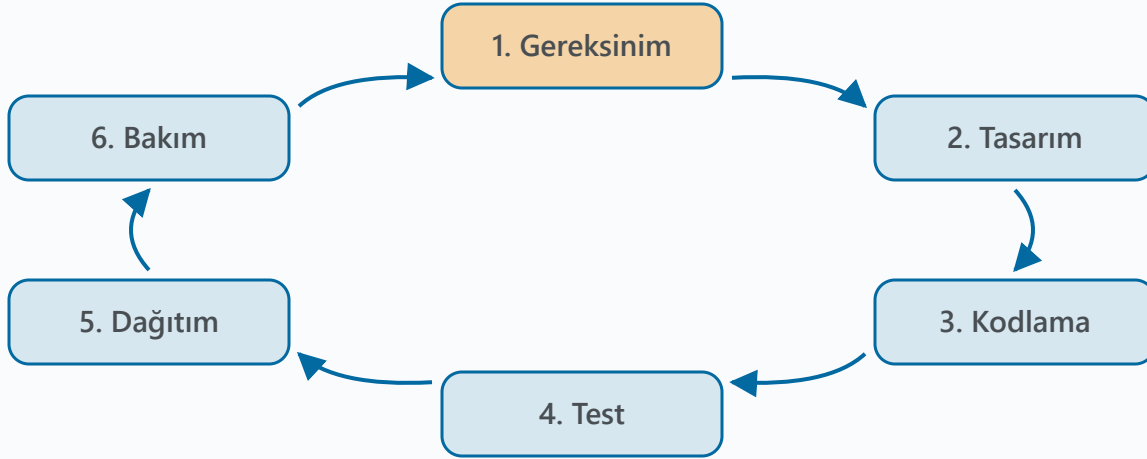
Geçersiz ölçü

Akış Diyagramı

- Algoritmadaki işlem, karar ve tekrarları standart simgelerle gösterir.
- Karmaşık dallanmaları kod yazmadan önce görünür kılar.
- Diyagram okunabilir kaldığı sürece iletişim ve gözden geçirmeyi kolaylaştırır.
- Ayrıntılar: [Akış Diyagramları](#) sunumu.



Yazılım Geliştirme Süreci

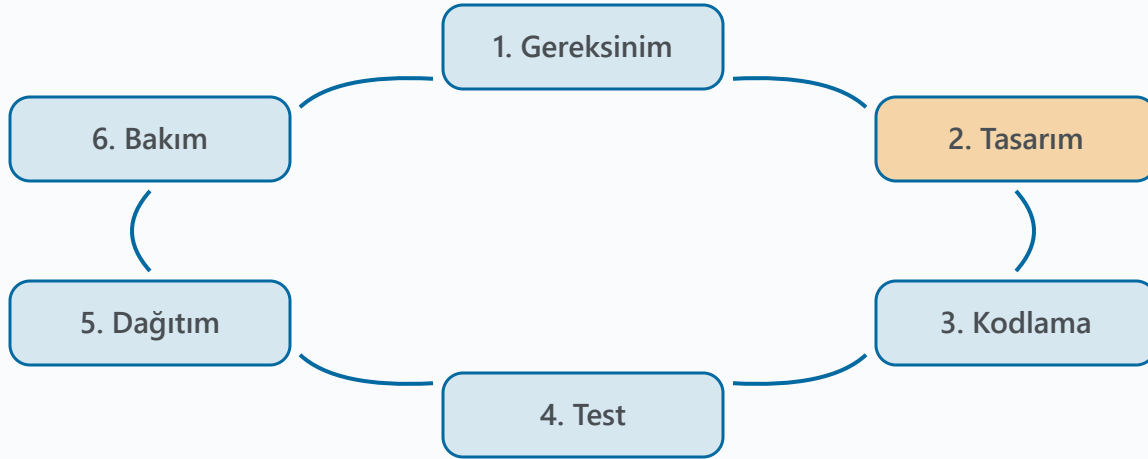


Gereksinim: kullanıcı ne istiyor? Örn. "öğrenci not ortalamasını hesaplayan uygulama".

Adım 1 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Yazılım Geliştirme Süreci

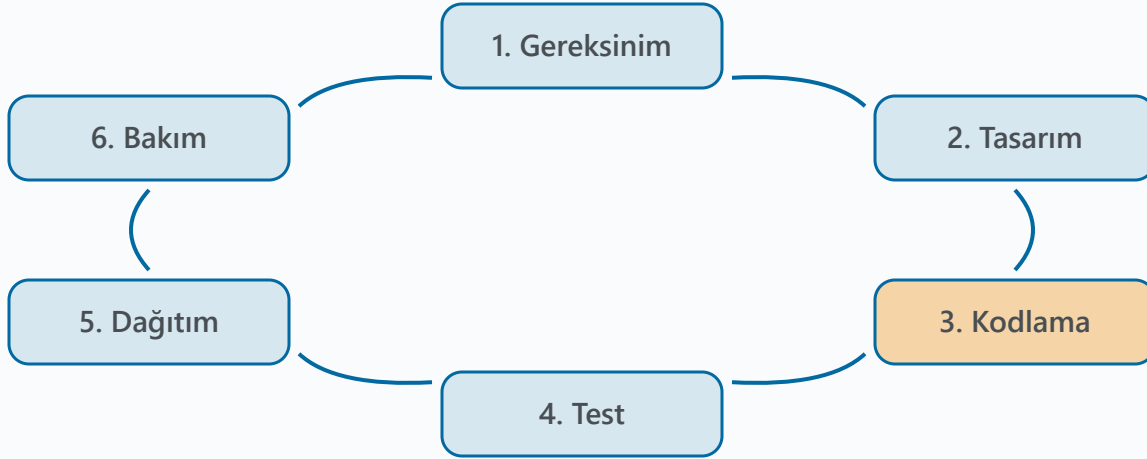


Tasarım: ekranlar, veri ve algoritmalar planlanır (akış diyagramı burada kullanılır).

Adım 2 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Yazılım Geliştirme Süreci

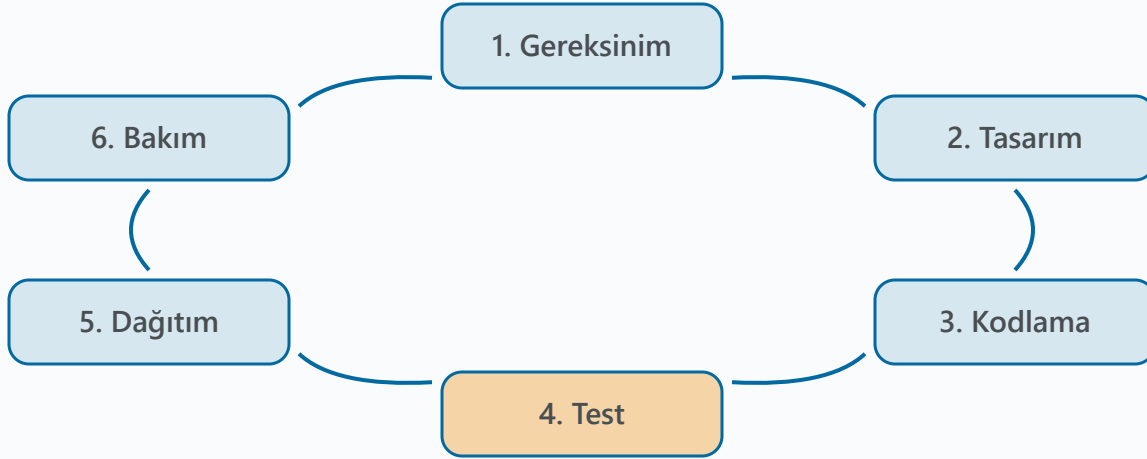


Kodlama: tasarım bir programlama diliyle yazılır.

Adım 3 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Yazılım Geliştirme Süreci

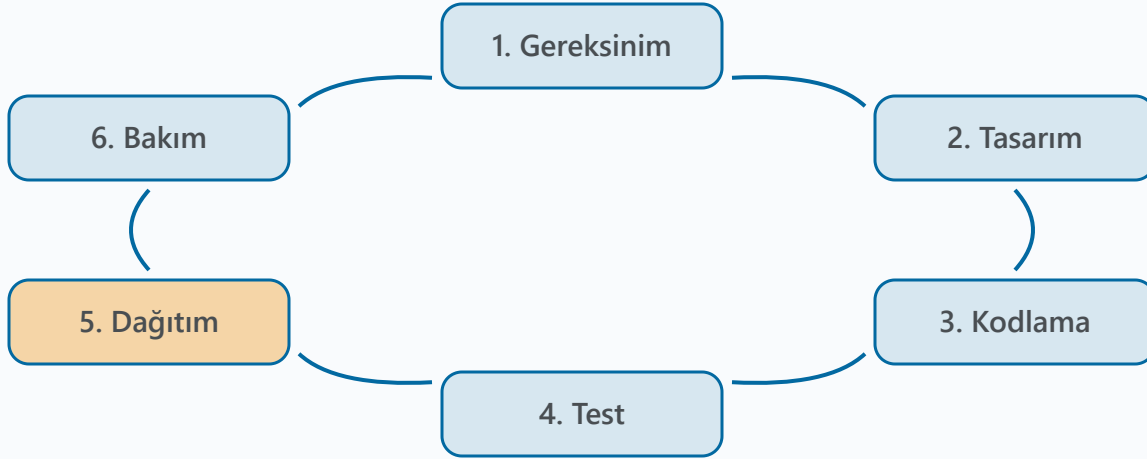


Test: beklenen ve gerçek sonuçlar karşılaştırılır; hata varsa kodlamaya geri dönülür.

Adım 4 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Yazılım Geliştirme Süreci

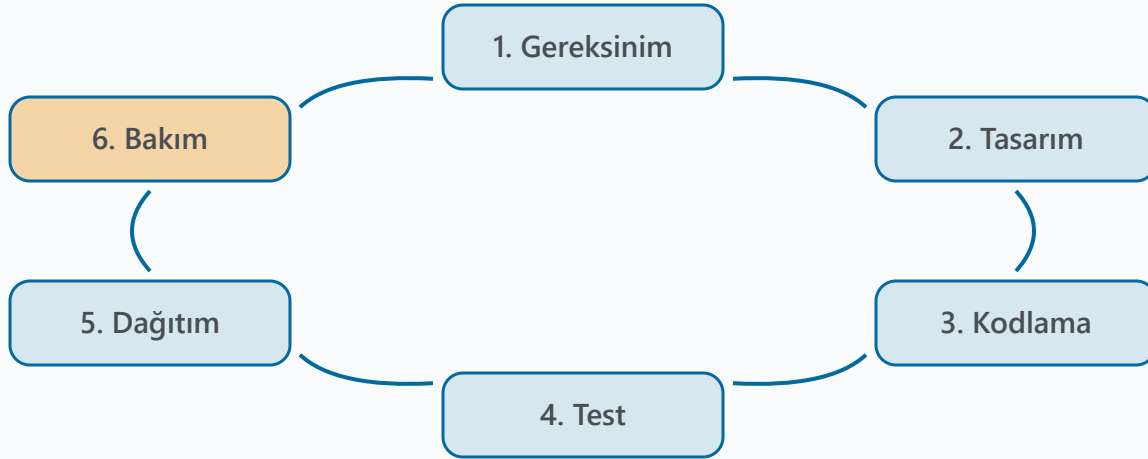


Dağıtım: program kullanıcıya ulaştırılır (kurulum, yayın).

Adım 5 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Yazılım Geliştirme Süreci



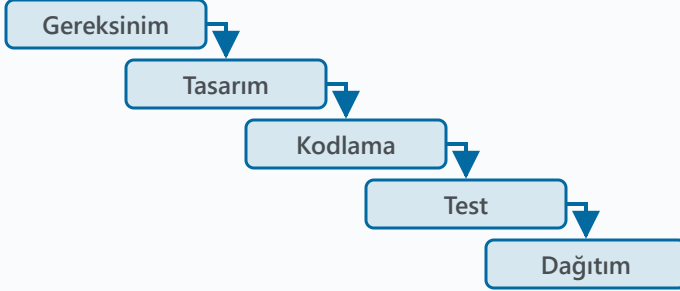
Bakım: hatalar giderilir, yeni istekler gelir → yeni gereksinimlerle döngü yeniden başlar.

Adım 6 / 6

- Aşamalar projeye göre ardışık veya yinelemeli yürütülebilir; kullanıcı geri bildirimi gereksinimi ve tasarımı değiştirebilir.

Geliştirme Yaklaşımları

Şelale



Çevik



her döngü sonunda çalışan sürüm + geri bildirim

Yaklaşım	Güçlü yön	Dikkat
Şelale modeli	Aşamalar ve belgeler nettir	Değişikliğe dönüş maliyetlidir
Çevik	Kısa döngülerle geri bildirim alır	Sürekli iletişim gerektirir
Prototipleme	Belirsiz gereksinimi görünür kılar	Prototip ürün sanılmamalıdır

Geliştirme Araçları

Kod düzenleyici / IDE

Kod yazma, tamamlama ve proje yönetimi.

Derleyici / çalışma ortamı

Programı çevirme ve çalıştırma.

Hata ayıklayıcı

Program durumunu adım adım inceleme.

Sürüm kontrolü

Değişiklik geçmişi ve ekip çalışması.

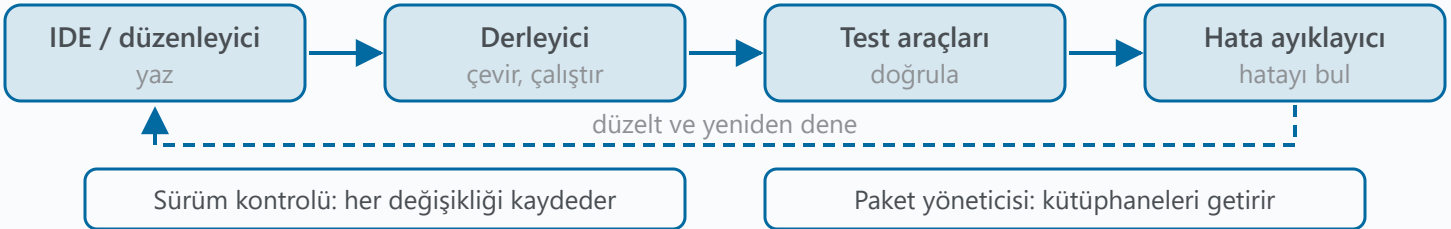
Test araçları

Beklenen davranışı otomatik doğrulama.

Paket yöneticisi

Bağımlılıkları yönetme.

Bir geliştirme turu: her araç döngünün farklı adımında devreye girer



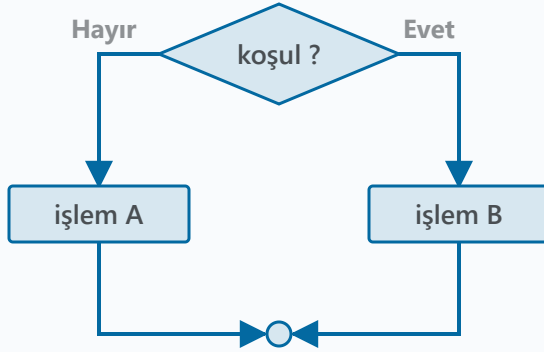
Değişkenler ve Veri Tipleri

- Değişken, bellekte bir değeri tutan **adlandırılmış kutudur**; program boyunca değeri değişebilir.
- Veri tipi değerin temsilini ve üzerinde yapılabilecek işlemleri sınırlar.
- Anlamlı adlar programın amacını görünür kılar.

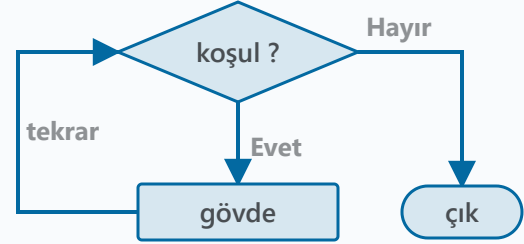
yas	boy	ad	gecti
19	1.75	"Ali"	doğru
tip: tam sayı	tip: ondalık sayı	tip: dizgi	tip: mantıksal

Java'daki karşılıkları (int, double, String, boolean) Bölüm 6'da.

Karar ve Döngü



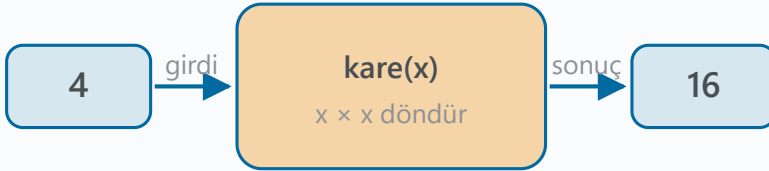
Karar: iki yoldan yalnızca biri



Döngü: geri dönen ok

- Karar yapısı koşula göre farklı işlem yollarından birini seçer; döngü bir işlem grubunu koşul boyunca tekrarlar.
- Her ikisinde de koşulun ne zaman değiştiği açık olmalıdır (yoksa sonsuz döngü).

Fonksiyonlar ve Diziler



notlar dizisi

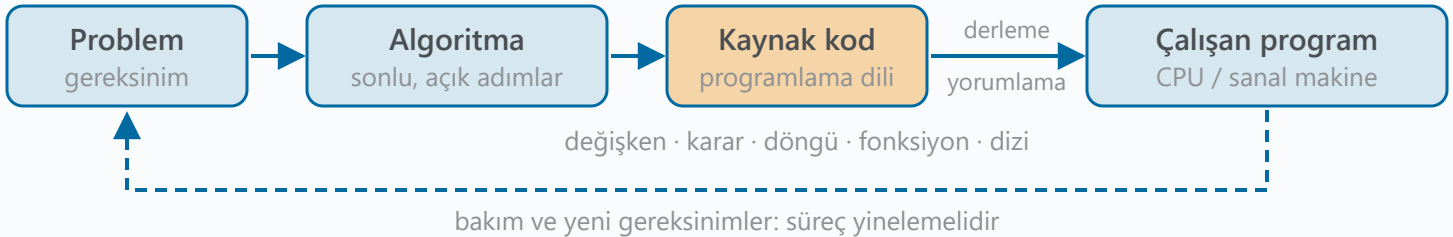
85	72	90	64
[0]	[1]	[2]	[3]

indis 0'dan başlar

- Fonksiyon belirli görevi adlandırır, girdi alabilir ve sonuç döndürebilir.
- Dizi aynı tipteki çok sayıda değeri indisle saklar.
- Bu yapılar programı küçük, test edilebilir parçalara ayırmaya yardımcı olur (Bölüm 9 ve 10).

Bölüm Özeti

- Programlama dili algoritmayı bilgisayarın yürütebileceği biçimde ifade eder.
- Kaynak kod derleme, yorumlama veya hibrit bir süreçle çalıştırılır.
- Yazılım geliştirme gereksinimden bakıma uzanan yinelemeli bir süreçtir.
- Değişken, karar, döngü, fonksiyon ve dizi temel programlama yapılarını oluşturur.



Kontrol Soruları

1. Kaynak kod ile makine dili arasındaki fark nedir?
2. Derlenen ve yorumlanan programlar nasıl ayrılır?
3. Bir algoritma neden programlama dilinden bağımsız düşünülebilir?
4. Yazılım testinin amacı nedir?
5. Karar yapısı ile döngünün görevleri nasıl farklıdır?

Alıştırmalar

1. Aşağıdaki hataları derleme, çalışma zamanı ve mantık hatası olarak sınıflandırın: noktalı virgül unutmak; ortalama için toplamı n yerine $n + 1$ ile bölmek; sıfıra bölmek.
2. "Bir öğrencinin üç sınav notunun ortalamasını hesapla" problemi için girdi, çıktı ve kısıtları yazın.
3. Aynı problem için sözde kod yazın ve (70, 85, 90) girdisiyle izleyin; sonuç 81.67 olmalı.
4. Dikdörtgen alanı algoritmasına çevre hesabını ekleyin: $\text{cevre} = 2 \times (\text{en} + \text{boy})$. $\text{en} = 3$, $\text{boy} = 4$ için beklenen çıktılar nedir?
5. Bir telefon uygulaması için yazılım geliştirme sürecinin her aşamasında yapılacak birer işi yazın.
6. Java, Python ve C dillerini derlenen / yorumlanan / hibrit olarak sınıflandırın ve çalıştırma hatlarını çizin.

► İpuçları

Son

Programlamaya Giriş

Sonraki sunum: [Algoritmalar](#) · [Çalışma soruları](#) · [Terim sözlüğü](#)